Aplicação de Derivadas no Jogo - MagicArcor

MODELAGEM MATEMÁTICA E FUNÇÕES APLICADAS AO JOGO

1.OBJETIVO

Neste trabalho usamos derivadas para descrever como as coisas **mudam ao longo do tempo** no nosso jogo: o quanto o personagem anda por segundo, quão rápido o BLOCK acelera no ataque, quão rápido a vida do jogador cai a cada turno, quanto o bônus de pontos diminui e como a dificuldade sobe de fase em fase. Em todos os casos, a derivada responde à mesma pergunta: **qual é a taxa de variação neste instante?**

Os valores numéricos usados nos exemplos são **modelos de projeto** coerentes com o jogo (por exemplo, vida do jogador 50, ataque 10 e defesa 4, como no nosso GDD). Eles podem ser ajustados no Inspector da Unity sem mudar a ideia matemática

2.CONCEITO E INTERPRETAÇÃO GEOMÉTRICA

A **taxa de variação média** de uma função f entre x e x + h é a inclinação da reta secante que passa pelos dois pontos do gráfico. Quando h fica cada vez menor, essa reta se aproxima da **reta tangente**. A inclinação da tangente é a derivada:

$$f'(x) = \lim_{h \to 0} \; [\, f(x + h) - f(x) \,] / h$$

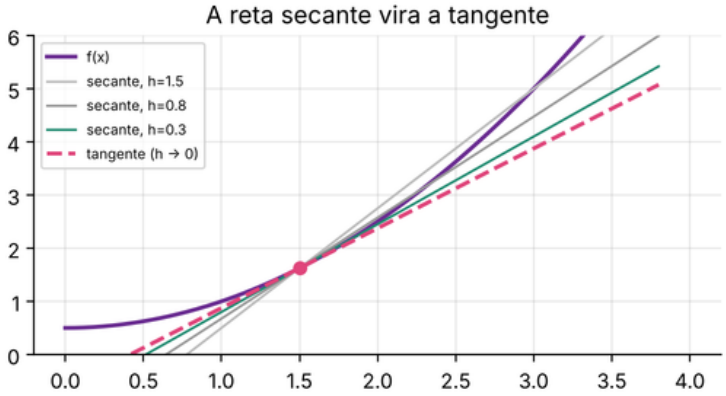


Figura 1 – As retas secantes (cinza e verde) se aproximam da tangente (rosa) quando h diminui.

Leitura geométrica no jogo:

O que vemos no gráfico	O que a derivada diz	Efeito na jogabilidade
Tangente subindo (f' > 0)	A grandeza está aumentando	Dificuldade subindo, pontos crescendo, personagem avançando
Tangente descendo (f' < 0)	A grandeza está diminuindo	Vida caindo, bônus de tempo diminuindo, distância diminuindo
Tangente horizontal (f' = 0)	Ponto de máximo, mínimo ou parado	Personagem parado, pico da curva de dificuldade
Tangente muito inclinada (|f'| grande)	Mudança rápida	Sensação de pressa, perigo, ataque veloz
Tangente pouco inclinada (|f'| pequena)	Mudança lenta	Sensação de calma, jogo previsível

Regras usadas: $(c)' = 0$; $(a \cdot t^n)' = a \cdot n \cdot t^{n-1}$; $(e^{kt})' = k \cdot e^{kt}$; a derivada da soma é a soma das derivadas. A derivada da derivada (segunda derivada) mede a **aceleração** da mudança.

3. APLICAÇÃO 1: VELOCIDADE E ACELERAÇÃO DO MOVIMENTO

3.1 Personagem andando

Quando o jogador segura W, ele anda com velocidade constante, por exemplo 4 m/s. A posição é s(t) = 4t. Derivando: **s'(t) = 4**, ou seja, a derivada é constante e a tangente coincide com a própria reta. Isso significa que a movimentação é previsível e fácil de controlar.

Na Unity, o nosso script de animação já faz uma versão da derivada: ele calcula a velocidade pela mudança de posição entre dois quadros (delta de posição dividido pelo tempo), e usa esse valor para decidir entre as animações Parado e Andando. Isso é a taxa de variação média em um intervalo muito pequeno (h = Time.deltaTime).

3.2 Avanço acelerado do BLOCK (modelo)

O BLOCK é gigante e pesado: não sai correndo, ele **acelera**. Modelamos a posição do avanço por s(t) = $1{,}5 \cdot t^2$ (metros). Então:

$$v(t) = s'(t) = 3t \qquad a(t) = v'(t) = s''(t) = 3 \text{ m/s}^2$$

Em t = 2 s: posição 6 m, velocidade 6 m/s. A inclinação da tangente no gráfico da posição é exatamente 6, e ela aumenta com o tempo: o avanço começa lento e termina forte. Para o jogador, isso significa um **aviso de reação**: dá tempo de se preparar para defender.

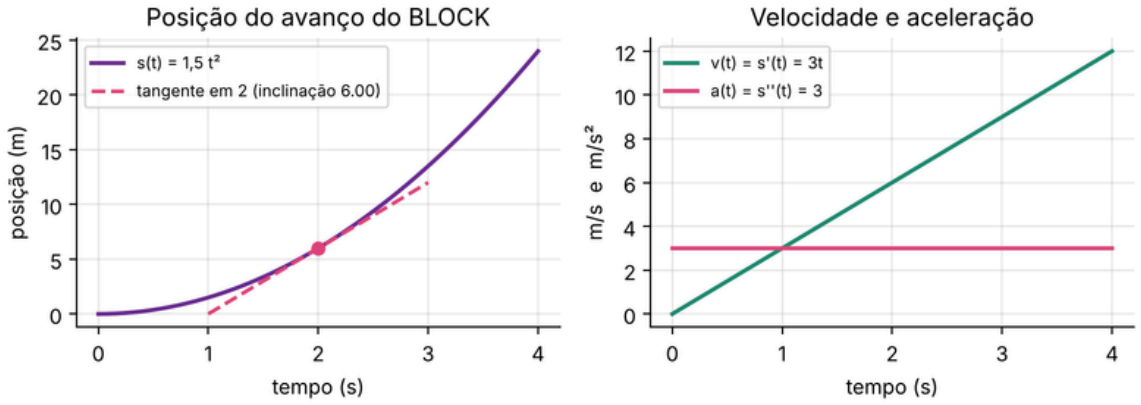


Figura 2 – Posição, velocidade e aceleração do avanço do BLOCK (modelo).

3.3 Tortuguita seguindo o jogador

No jogo, a Tortuguita segue o jogador e diminui a velocidade conforme chega perto (a velocidade é proporcional à distância que falta, com um limite máximo). Em termos de cálculo, se d(t) é a distância até o jogador e a distância desejada é 2 m, a regra "velocidade = 4 · (distância − 2)" vira a equação d'(t) = −4(d − 2), cuja solução, partindo de 6 m, é:

$$d(t) = 2 + 4 \cdot e^{-4t} \qquad d'(t) = -16 \cdot e^{-4t}$$

A derivada é negativa (a distância diminui) e vai para perto de zero: a Tortuguita chega rápido e **desacelera suavemente** em vez de bater no jogador. O limite de velocidade do script (4 m/s) corta o pico do começo (linha tracejada), por isso ela não "dispara".

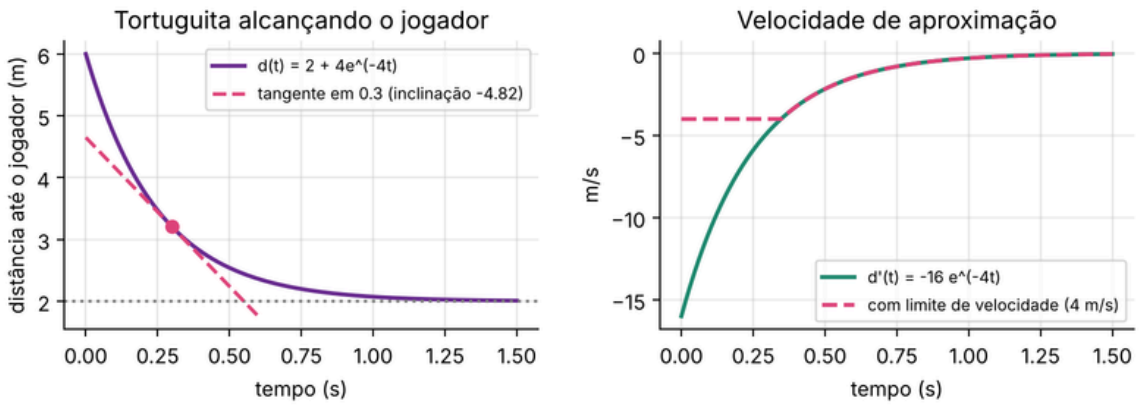


Figura 3 – Distância e velocidade de aproximação da Tortuguita.

4. APLICAÇÃO 2: VIDA E COMBATE POR TURNOS

Em um RPG por turnos, o tempo é medido em **turnos** (n). Seja V(n) a vida do jogador depois de n ataques do inimigo. Com vida inicial 50, se cada golpe tira em média 6 pontos, V(n) = 50 − 6n, logo **V'(n) = −6** pontos por turno. Se o jogador defende (a defesa reduz o dano pela metade neste exemplo), V(n) = 50 − 3n e **V'(n) = −3**.

O número de turnos que o jogador aguenta é V(n) = 0, ou seja, n = 50/6 ≈ 8,3 sem defender e n = 50/3 ≈ 16,7 defendendo. Defender não muda o ponto de partida, muda a inclinação da reta: é a derivada que dá o poder estratégico da defesa. No nosso jogo, a janela de defesa reativa do turno do inimigo existe justamente para o jogador trocar a inclinação −6 por −3 no momento certo _

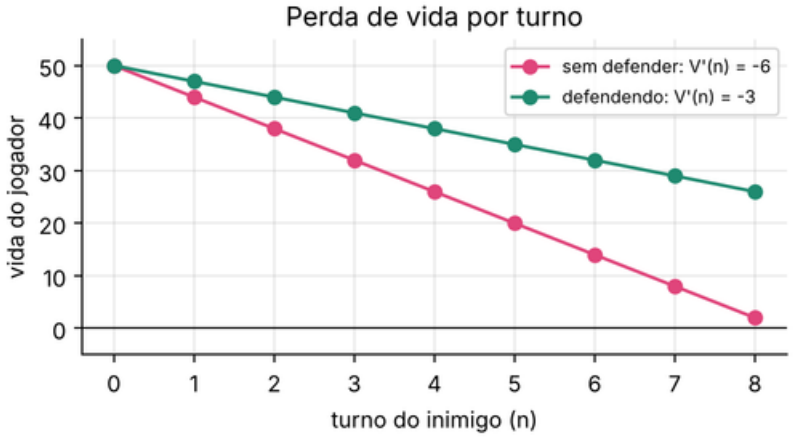


Figura 4 – A defesa faz a tangente ficar menos inclinada (a vida cai mais devagar).

Situação	V(n)	V'(n) (vida por turno)	Turnos até V = 0
Sem defender	50 6n	−6	≈ 8,3
Defendendo	50 − 3n	−3	≈16,7

5. Aplicação 3: pontuação e bônus de tempo no labirinto

IImagine que o labirinto dá um bônus de pontos que diminui com o tempo gasto: B(t) = 1000·e−t/60 (t em segundos). A taxa de variação é:

$$B'(t) = -(1000/60)\cdot e^{-t/60} \approx -16{,}7\cdot e^{-t/60}$$

Em t = 0, o jogador perde cerca de 16,7 pontos por segundo; em t = 30 s, cerca de 10,1 pontos por segundo; em t = 60 s, cerca de 6,1 pontos por segundo. A derivada é sempre negativa (o bônus só cai), mas seu módulo diminui: o jogo **pune forte no começo e perdoa depois**. Assim quem se perde no labirinto não é massacrado, o que combina com o clima de "tentativa e erro" das falas da Tortuguita.

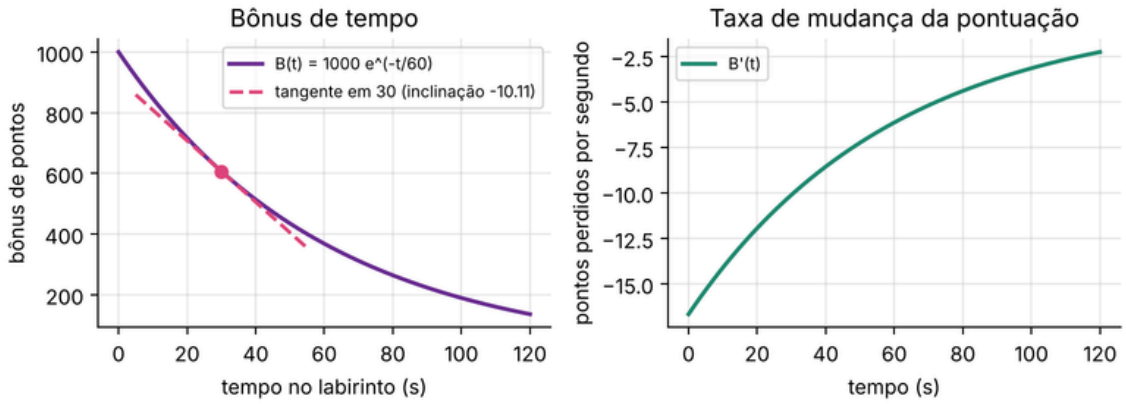


Figura 5 – Bônus de tempo B(t) e sua taxa de variação B'(t).

6. APLICAÇÃO 4: CURVA DE DIFICULDADE

Seja f o número da fase (0 = Fase 0 com o Ursinho, 1 = Labirinto e BLOCK, 2 = puzzle e Beijinho Gigante, e assim por diante) e D(f) uma nota de dificuldade (vida + ataque dos inimigos, por exemplo). Modelo:

$$D(f) = 12 + 6f + 3f^2 \qquad D'(f) = 6 + 6f \qquad D''(f) = 6$$

Fase f	D(f)	D'(f)	Leitura
0	12	6	Começa fácil e sobe devagar
1	21	12	BLOCK: a dificuldade sobe duas vezes mais rápido
2	36	18	Subida forte
3	57	24	Risco de frustração se não houver ajuda

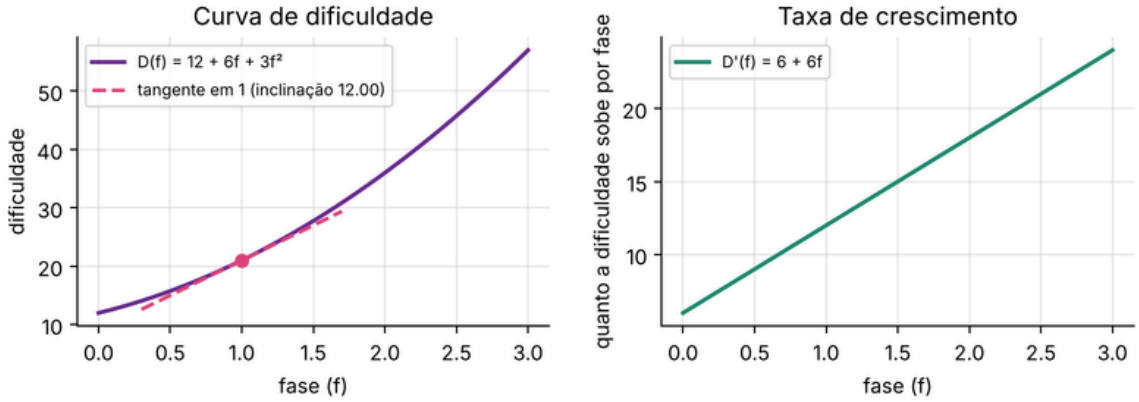


Figura 6 – Curva de dificuldade e sua taxa de crescimento.

Interpretação geométrica: a tangente da curva em f = 1 tem inclinação 12, ou seja, nesse ponto a dificuldade aumenta 12 "unidades" por fase. Como D'' = 6 > 0, a curva é côncava para cima: a dificuldade **acelera**. Para um jogo de estudantes e iniciantes isso é arriscado, então usamos a derivada como ferramenta de balanceamento: se D' cresce rápido demais, compensamos com

recompensas (Block Pistache, +30% de força e +50% de velocidade, e o item Chave), o que equivale a achatar a tangente. O ideal para o público é D' crescendo pouco, com uma curva quase linear.

7. Como a derivada aparece no código (Unity)

Na Unity não temos a fórmula pronta de tudo, então calculamos a derivada **numericamente**: variação dividida pelo tempo entre dois quadros.

```
// Velocidade = derivada numerica da posicao
Vector3 posAnterior;
void Update()
{
    Vector3 velocidade = (transform.position - posAnterior) / Time.deltaTime;
    float rapidez = new Vector2(velocidade.x, velocidade.z).magnitude;
    posAnterior = transform.position;
}

// Avanco acelerado do BLOCK: s(t) = 1.5 * t^2  (v = 3t, a = 3)
float t;
void Update()
{
    t += Time.deltaTime;
    transform.position = inicio + direcao * (1.5f * t * t);
}

// Dificuldade da fase: D(f) = 12 + 6f + 3f^2
float Dificuldade(int f) { return 12 + 6 * f + 3 * f * f; }
float TaxaDificuldade(int f) { return 6 + 6 * f; }   // D'(f)
```

8. CONCLUSÃO

A derivada transforma "o jogo está mais rápido, mais difícil ou mais perigoso" em números que podemos controlar. Geometricamente, ela é a inclinação da tangente: quanto maior, mais intensa a mudança; quando é zero, a grandeza para ou atinge um extremo; quando muda de sinal, o comportamento se inverte. Usamos isso para planejar o movimento dos personagens, o poder da defesa nos turnos, o bônus de tempo do labirinto e o ritmo com que o jogo fica mais difícil, mantendo a experiência desafiadora, mas justa.